Date :

10. Java arrays

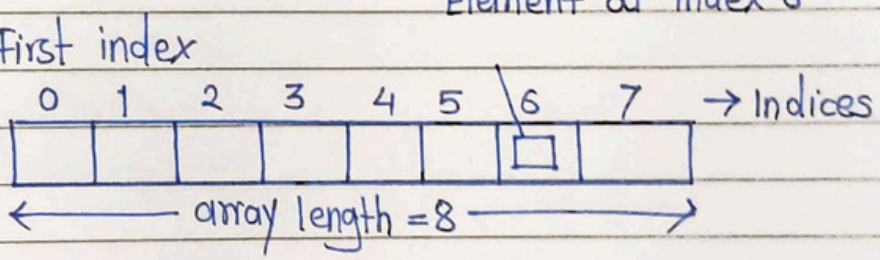
10.1 Single Dimension array

Normally, an array is a collection of similar type of elements which has contiguous memory location.

Java array is an object which contains elements of a similar data type. Additionally, the elements of an array are ~~sorted~~ stored in a contiguous memory allocation. It is a data structure where we store similar elements. We can store only a fixed set of elements in a java array.

Array in java is index-based, the first element of the array is stored at the 0th index, 2nd element is stored on 1st index.

Unlike C/C++, we get the length of the array using the length member. In C/C++, we need to use the sizeof operator.



Single dimensional array syntax:

```
dataType[] arr;
```

Instantiation:

```
arrayRefVar = new datatype[size];
```

Date :

Example:

```
// Java program to illustrate how to declare, instantiate, initialize
// and traverse the Java array
class Testarray {
public static void main(String args[]){
int a[] = new int[5]; //declaration and instantiation
 a[0] =10; // initialization
 a[1] = 20;
 a[2] = 70;
 a[3] = 40;
 a[4] = 50;
 // traversing array
 for(int i=0 ; i<a.length; i++)
 System.out.println(a[i]); //length is the property of array.
 }}
```

Output:

```
10
20
70
40
50
```

10.2 Multidimensional array

In such case, data is stored in row and column based index (also known as matrix form).

Syntax:

```
dataType[][] arrayRefVar;
```

Date :

Instantiation:

```
int[][] arr = new int[3][3]; // 3 rows and 3 column.
```

Example to initialize Multidimensional Array in Java:

```
arr[0][0]=1;
arr[0][1]=2;
arr[0][2]=3;
arr[1][0]=4;
arr[1][1]=5;
arr[1][2]=6;
arr[2][0]=7;
arr[2][1]=8;
arr[2][2]=9;
```

Example of multidimensional array:

```
// Java program to illustrate the use of multidimensional array
class Testarray3{
public static void main(String args[]){
// declaring and initializing 2D array
int arr[][] = {{1,2,3},{2,4,5},{4,4,5}};
// printing 2D array
for(int i=0;i<3;i++){
for(int j=0;j<3;j++){
  System.out.println(arr[i][j]+" ");
 }
System.out.println();
}
}}
```

Date :

Output :

```
1 2 3
2 4 5
4 4 5
```

10.3 cloning an Array in Java :

Since, Java array implements the cloneable interface we can create the clone of the Java array. If we create the clone of a single-dimensional array, it creates the deep copy of the java array. It means, it will copy the actual value. But, if we create the clone of a multidimensional array, it creates the shallow copy of the Java array which means it copies the references.

```
// Java program to clone the array
class Testarray1{
public static void main(String args[]){
int arr[] ={33,3,4,5};
System.out.println(" printing original array:");
for(int i:arr)
System.out.println(i);
system.out.println(" printing clone of the array:");
int carr[] = arr.clone();
for (int i: carr)
System.out.println(i);
System.out.println("Are both equal?");
System.out.println(arr==carr);
}}
```

Date :

Output:

```
printing original array:
33
3
4
5
printing clone of the array:
33
3
4
5
Are both equal?
false
```